

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 4
За седьмой семестр
По дисциплине: «Интеллектуальный анализ данных »
Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:
Студент 4 курса
Группы ИИ-26
Хомиченко И.А.
Проверил:
Никитин С.К.

Цель: научиться осуществлять предобучение нейронных сетей с помощью RBM

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.
2. Сравнить результаты, полученные при
 - обучении без предобучения (ЛР 3);
 - обучении с предобучением, используя автоэнкодерный подход (ЛР3);
 - обучении с предобучением, используя RBM.
3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;
4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Тип задачи	Целевая переменная
7	https://archive.ics.uci.edu/dataset/503/hepatitis+c+virus+hcv+for+egyptian+patients	классификация	Baselinehistological staging

Код программы:

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from tensorflow.keras import layers, Model
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.neural_network import BernoulliRBM
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
from ucimlrepo import fetch_ucirepo

np.random.seed(42)
tf.random.set_seed(42)

EPOCHS = 100
PRETRAIN_EPOCHS = 100
HIDDEN = [64, 32, 16]

def build_model(input_dim, classes):
```

```

model = tf.keras.Sequential([
    layers.Input(shape=(input_dim,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(classes if classes > 2 else 1,
                  activation="softmax" if classes > 2 else "sigmoid")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy" if classes > 2 else
"binary_crossentropy",
    metrics=["accuracy"]
)

return model

def ae_layer(X, units):
    inp = layers.Input(shape=(X.shape[1],))
    enc = layers.Dense(units, activation="relu")(inp)
    dec = layers.Dense(X.shape[1], activation="sigmoid")(enc)

    ae = Model(inp, dec)
    encoder = Model(inp, enc)

    ae.compile(optimizer="adam", loss="mse")
    ae.fit(X, X, epochs=PRETRAIN_EPOCHS, batch_size=32, verbose=0)

    return encoder

def rbm_layer(X, units):
    rbm = BernoulliRBM(
        n_components=units,
        learning_rate=0.01,
        batch_size=32,
        n_iter=1,
        random_state=42
    )

    for epoch in range(PRETRAIN_EPOCHS):
        rbm.partial_fit(X)

        if (epoch + 1) % 20 == 0:
            score = np.mean(rbm.score_samples(X))
            print(f" RBM epoch {epoch + 1}/{PRETRAIN_EPOCHS},
loss={score:.4f}")

    return rbm

def set_rbm_weights(layer, rbm):
    layer.set_weights([
        rbm.components_.T.astype(np.float32),
        rbm.intercept_hidden_.astype(np.float32)
    ])

def train_three(X_train, y_train, X_test, classes, name):
    print(f"\n--- {name}: Baseline ---")

    tf.keras.backend.clear_session()

```

```

tf.random.set_seed(42)

baseline = build_model(X_train.shape[1], classes)
baseline.fit(X_train, y_train, epochs=EPOCHS, batch_size=32, verbose=0)

p = baseline.predict(X_test, verbose=0)
pred_base = np.argmax(p, axis=1) if classes > 2 else (p.ravel() >=
0.5).astype(int)

print(f"\n--- {name}: AE Pretraining ---")

X_temp = X_train.copy()
encoders = []

for units in HIDDEN:
    encoder = ae_layer(X_temp, units)
    encoders.append(encoder)
    X_temp = encoder.predict(X_temp, verbose=0)

tf.keras.backend.clear_session()
tf.random.set_seed(42)

ae_model = build_model(X_train.shape[1], classes)

for i in range(3):
    ae_model.layers[i].set_weights(encoders[i].get_weights())

ae_model.fit(X_train, y_train, epochs=EPOCHS, batch_size=32, verbose=0)

p = ae_model.predict(X_test, verbose=0)
pred_ae = np.argmax(p, axis=1) if classes > 2 else (p.ravel() >=
0.5).astype(int)

print(f"\n--- {name}: RBM Pretraining ---")

X_temp = X_train.copy()
rbms = []

for units in HIDDEN:
    rbm = rbm_layer(X_temp, units)
    rbms.append(rbm)
    X_temp = rbm.transform(X_temp)

tf.keras.backend.clear_session()
tf.random.set_seed(42)

rbm_model = build_model(X_train.shape[1], classes)

for i in range(3):
    set_rbm_weights(rbm_model.layers[i], rbms[i])

rbm_model.fit(X_train, y_train, epochs=EPOCHS, batch_size=32, verbose=0)

p = rbm_model.predict(X_test, verbose=0)
pred_rbm = np.argmax(p, axis=1) if classes > 2 else (p.ravel() >=
0.5).astype(int)

return pred_base, pred_ae, pred_rbm

def binary_result(y, pred, name):
    acc = accuracy_score(y, pred)
    prec = precision_score(y, pred, zero_division=0)
    rec = recall_score(y, pred, zero_division=0)

```

```

f1 = f1_score(y, pred, zero_division=0)

print(
    f"[{name}] "
    f"Acc={acc:.4f} "
    f"Prec={prec:.4f} "
    f"Rec={rec:.4f} "
    f"F1={f1:.4f}"
)

return [acc, prec, rec, f1]

def multi_result(y, pred, name):
    acc = accuracy_score(y, pred)
    flm = f1_score(y, pred, average="macro", zero_division=0)
    flw = f1_score(y, pred, average="weighted", zero_division=0)

    print(
        f"[{name}] "
        f"Acc={acc:.4f} "
        f"F1_macro={flm:.4f} "
        f"F1_weighted={flw:.4f}"
    )

    return [acc, flm, flw]

print("# ЧАСТЬ A. HCV")

hcv = fetch_ucirepo(id=503)

X_hcv = hcv.data.features.copy()
y_hcv = hcv.data.targets["Baselinehistological staging"].values.ravel()

X_hcv = X_hcv.apply(pd.to_numeric, errors="coerce")
X_hcv = X_hcv.fillna(X_hcv.median(numeric_only=True)).fillna(0)

y_hcv = LabelEncoder().fit_transform(y_hcv.astype(str))

Xh_train, Xh_test, yh_train, yh_test = train_test_split(
    X_hcv,
    y_hcv,
    test_size=0.2,
    random_state=42,
    stratify=y_hcv
)

scaler = MinMaxScaler()

Xh_train = scaler.fit_transform(Xh_train).astype(np.float32)
Xh_test = scaler.transform(Xh_test).astype(np.float32)

Xh_train = np.clip(Xh_train, 0, 1)
Xh_test = np.clip(Xh_test, 0, 1)

h_base, h_ae, h_rbm = train_three(
    Xh_train,
    yh_train,
    Xh_test,
    4,
    "HCV"
)

```

```

print("\n--- HCV: Результаты ---")

hm_base = multi_result(yh_test, h_base, "HCV Baseline")
hm_ae = multi_result(yh_test, h_ae, "HCV AE")
hm_rbm = multi_result(yh_test, h_rbm, "HCV RBM")

hcv_table = pd.DataFrame({
    "Метрика": ["Accuracy", "F1_macro", "F1_weighted"],
    "Baseline": hm_base,
    "AE": hm_ae,
    "RBM": hm_rbm
})

hcv_table["Δ_AE"] = hcv_table["AE"] - hcv_table["Baseline"]
hcv_table["Δ_RBM"] = hcv_table["RBM"] - hcv_table["Baseline"]

print("\n--- HCV: Сравнение ---")
print(hcv_table.round(4).to_string(index=False))

print("\n# ЧАСТЬ B. MUSHROOM")

url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/mushroom/"
    "agaricus-lepiota.data"
)

columns = [
    "class", "cap-shape", "cap-surface", "cap-color", "bruises",
    "odor", "gill-attachment", "gill-spacing", "gill-size",
    "gill-color", "stalk-shape", "stalk-root",
    "stalk-surface-above-ring", "stalk-surface-below-ring",
    "stalk-color-above-ring", "stalk-color-below-ring",
    "veil-type", "veil-color", "ring-number", "ring-type",
    "spore-print-color", "population", "habitat"
]

mush = pd.read_csv(url, header=None, names=columns)

X_mush = mush.drop(columns=["class"])
y_mush = (mush["class"] == "p").astype(int).values

Xm_train, Xm_test, ym_train, ym_test = train_test_split(
    X_mush,
    y_mush,
    test_size=0.2,
    random_state=42,
    stratify=y_mush
)

try:
    encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
except TypeError:
    encoder = OneHotEncoder(handle_unknown="ignore", sparse=False)

Xm_train = encoder.fit_transform(Xm_train).astype(np.float32)
Xm_test = encoder.transform(Xm_test).astype(np.float32)

m_base, m_ae, m_rbm = train_three(
    Xm_train,
    ym_train,
    Xm_test,
    2,

```

```

        "Mushroom"
    )

print("\n--- Mushroom: Результаты ---")

mm_base = binary_result(ym_test, m_base, "Mushroom Baseline")
mm_ae = binary_result(ym_test, m_ae, "Mushroom AE")
mm_rbm = binary_result(ym_test, m_rbm, "Mushroom RBM")

mush_table = pd.DataFrame({
    "Метрика": ["Accuracy", "Precision", "Recall", "F1"],
    "Baseline": mm_base,
    "AE": mm_ae,
    "RBM": mm_rbm
})

mush_table["Δ_AE"] = mush_table["AE"] - mush_table["Baseline"]
mush_table["Δ_RBM"] = mush_table["RBM"] - mush_table["Baseline"]

print("\n--- Mushroom: Сравнение ---")
print(mush_table.round(4).to_string(index=False))

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

hcv_preds = [h_base, h_ae, h_rbm]
mush_preds = [m_base, m_ae, m_rbm]
names = ["Baseline", "AE", "RBM"]

for i in range(3):
    sns.heatmap(
        confusion_matrix(yh_test, hcv_preds[i]),
        annot=True,
        fmt="d",
        cmap="Blues",
        ax=axes[0, i]
    )

    axes[0, i].set_title(f"HCV - {names[i]}")
    axes[0, i].set_xlabel("Predicted")
    axes[0, i].set_ylabel("Actual")

    sns.heatmap(
        confusion_matrix(ym_test, mush_preds[i]),
        annot=True,
        fmt="d",
        cmap="Blues",
        ax=axes[1, i],
        xticklabels=["Edible", "Poisonous"],
        yticklabels=["Edible", "Poisonous"]
    )

    axes[1, i].set_title(f"Mushroom - {names[i]}")
    axes[1, i].set_xlabel("Predicted")
    axes[1, i].set_ylabel("Actual")

plt.tight_layout()
plt.savefig("comparison.png", dpi=150)
plt.show()

print("\nВЫВОДЫ")

print("\nHCV:")

```

```

print(f"Baseline Fl_macro={hm_base[1]:.4f}")
print(f"AE Fl_macro={hm_ae[1]:.4f} (Δ={hm_ae[1] - hm_base[1]:+.4f})")
print(f"RBM Fl_macro={hm_rbm[1]:.4f} (Δ={hm_rbm[1] - hm_base[1]:+.4f})")

print("\nMUSHROOM:")
print(f"Baseline Fl={mm_base[3]:.4f}")
print(f"AE Fl={mm_ae[3]:.4f} (Δ={mm_ae[3] - mm_base[3]:+.4f})")
print(f"RBM Fl={mm_rbm[3]:.4f} (Δ={mm_rbm[3] - mm_base[3]:+.4f})")

```

Результат программы:

ЧАСТЬ А. HCV

--- HCV: Baseline ---

WARNING:tensorflow:From

C:\Users\fhdgsjfbdjsg\Desktop\иад4лаба\.venv\Lib\site-

packages\keras\src\backend\common\global_state.py:82: The name

tf.reset_default_graph is deprecated. Please use tf.compat.v1.reset_default_graph instead.

WARNING:tensorflow:TensorFlow GPU support is not available on native

Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

--- HCV: AE Pretraining ---

--- HCV: RBM Pretraining ---

RBM epoch 20/100, loss=-28.4426

RBM epoch 40/100, loss=-65.3640

RBM epoch 60/100, loss=-114.0625

RBM epoch 80/100, loss=-163.7111

RBM epoch 100/100, loss=-212.8033

RBM epoch 20/100, loss=-0.1869

RBM epoch 40/100, loss=-0.0003

RBM epoch 60/100, loss=-0.0000

RBM epoch 80/100, loss=-0.0000

RBM epoch 100/100, loss=-0.0000

RBM epoch 20/100, loss=-1.9231
RBM epoch 40/100, loss=-0.0720
RBM epoch 60/100, loss=-0.0027
RBM epoch 80/100, loss=-0.0001
RBM epoch 100/100, loss=-0.0000

--- HCV: Результаты ---

[HCV Baseline] Acc=0.2635 F1_macro=0.2618 F1_weighted=0.2631
[HCV AE] Acc=0.2527 F1_macro=0.2491 F1_weighted=0.2499
[HCV RBM] Acc=0.2563 F1_macro=0.1020 F1_weighted=0.1046

--- HCV: Сравнение ---

Метрика	Baseline	AE	RBM	Δ_{AE}	Δ_{RBM}
Accuracy	0.2635	0.2527	0.2563	-0.0108	-0.0072
F1_macro	0.2618	0.2491	0.1020	-0.0127	-0.1598
F1_weighted	0.2631	0.2499	0.1046	-0.0132	-0.1585

ЧАСТЬ B. MUSHROOM

--- Mushroom: Baseline ---

--- Mushroom: AE Pretraining ---

--- Mushroom: RBM Pretraining ---

RBM epoch 20/100, loss=-123.4237
RBM epoch 40/100, loss=-250.3736
RBM epoch 60/100, loss=-390.6710
RBM epoch 80/100, loss=-530.8775
RBM epoch 100/100, loss=-671.4515

RBM epoch 20/100, loss=-0.1596
 RBM epoch 40/100, loss=-0.0002
 RBM epoch 60/100, loss=-0.0000
 RBM epoch 80/100, loss=-0.0000
 RBM epoch 100/100, loss=-0.0000
 RBM epoch 20/100, loss=-1.8202
 RBM epoch 40/100, loss=-0.0629
 RBM epoch 60/100, loss=-0.0021
 RBM epoch 80/100, loss=-0.0001
 RBM epoch 100/100, loss=-0.0000

--- Mushroom: Результаты ---

[Mushroom Baseline] Acc=1.0000 Prec=1.0000 Rec=1.0000 F1=1.0000

[Mushroom AE] Acc=1.0000 Prec=1.0000 Rec=1.0000 F1=1.0000

[Mushroom RBM] Acc=1.0000 Prec=1.0000 Rec=1.0000 F1=1.0000

--- Mushroom: Сравнение ---

Метрика	Baseline	AE	RBM	Δ_{AE}	Δ_{RBM}
Accuracy	1.0	1.0	1.0	0.0	0.0
Precision	1.0	1.0	1.0	0.0	0.0
Recall	1.0	1.0	1.0	0.0	0.0
F1	1.0	1.0	1.0	0.0	0.0

ВЫВОДЫ

HCV:

Baseline F1_macro=0.2618

AE F1_macro=0.2491 ($\Delta=-0.0127$)

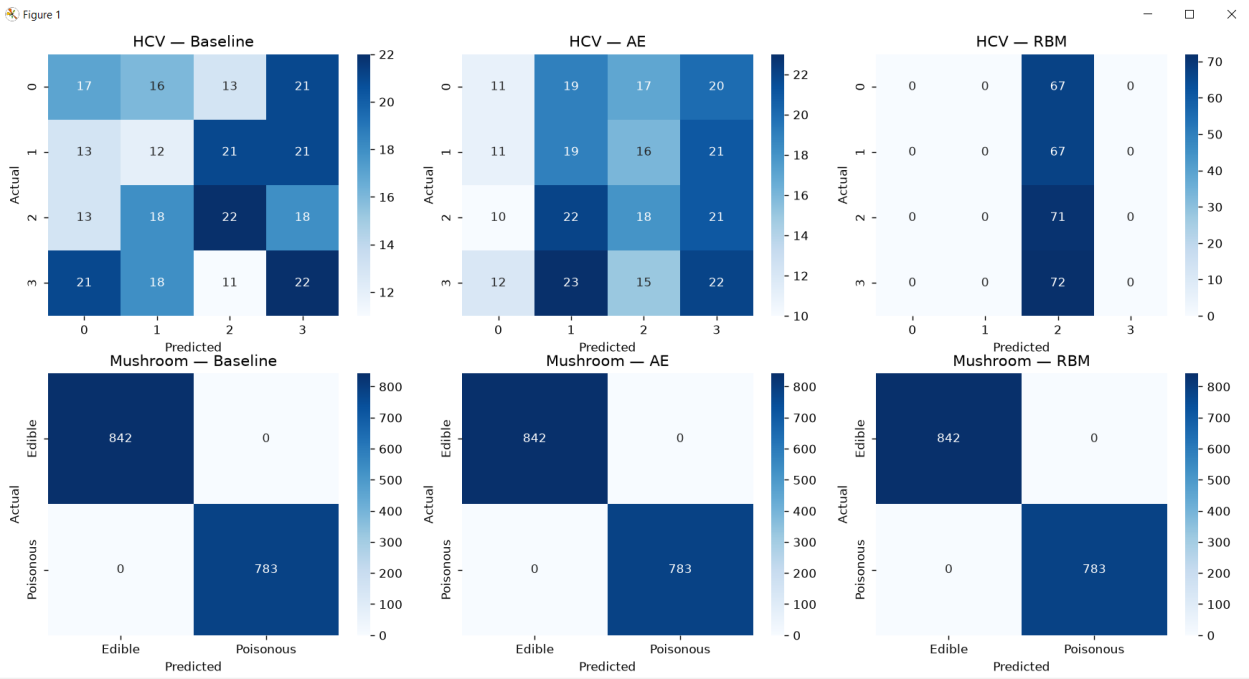
RBM F1_macro=0.1020 ($\Delta=-0.1598$)

MUSHROOM:

Baseline F1=1.0000

AE F1=1.0000 ($\Delta=+0.0000$)

RBM F1=1.0000 ($\Delta=+0.0000$)



Вывод: научился осуществлять предобучение нейронных сетей с помощью RBM